



PROJET FIL ROUGE 2

UPSSIBOT

Systèmes Robotiques & Interactifs (SRI 3A)

Équipe Projet :

Abdelhafidh GHOUILEM
Arthus DORIATH
Joan BELUSCA
Victor CHALUMEAUX

Livrable :

Rapport Technique PFR2

Client :

Julien Vanderstraeten

Repository Github

Table des matières

1	Architecture Globale du Système	3
1.1	Vision d'ensemble	3
1.2	Schéma de communication et Flux de données	3
1.3	La Boîte Noire du Parsing	4
1.4	Modes de fonctionnement	4
2	Communication Réseau et Serveur TCP	5
3	Arduino (Machine à états)	6
3.1	Sécurité et Watchdog	7
3.2	Odométrie Temporelle	7
3.3	La Calibration	7
4	Perception et Cartographie : Le Système LIDAR	8
4.1	Acquisition des données : La bibliothèque RPLidar	8
4.1.1	Transformation en Coordonnées Cartésiennes	9
4.2	L'Algorithme <code>makeClusters</code>	9
4.2.1	Logique de regroupement	9
4.3	Analyse et Reconnaissance de Formes	10
4.4	Caméra et reconnaissance d'images	10
4.4.1	Fonctionnement de <code>libcamera</code>	10
4.4.2	Problématique de <code>libcamera</code>	10
4.4.3	Solutions et architecture du module caméra	11
5	Multithreading et Architecture Parallèle	12
5.1	Problématique de la boucle unique (Monothreading)	12
5.2	Architecture Multi-threads et passage en C++	12
5.3	Synchronisation et Communication Inter-threads	13
5.3.1	Structure BiChannel et Files FIFO	13
5.4	Implémentation et Intégration	14
5.4.1	Sensor Task	14
5.4.2	Network Task	14
5.4.3	Control Task	14
6	GUI - Graphical User Interface	16
6.1	Architecture GUI	16
6.2	Les différentes Fenêtres	17
6.2.1	Fenêtre Principal	17
6.2.2	Menu Burger	18

6.2.3	Overlay	18
6.3	Gestion/Affichage des entités	19
6.3.1	Entités	19
6.3.2	Canvas de Simulation	20
6.4	Client TCP	21
6.5	Utils	21
6.5.1	Gestion de données	21
7	Bilan Personnel	22
7.1	Bilan de Abdelhafidh GHOUILEM	22
7.2	Bilan de Victor CHALUMEAUX	22
7.3	Bilan de Joan BELUSCA	23
7.4	Bilan d'Arthus DORIATH	23

Chapitre 1

Architecture Globale du Système

1.1 Vision d'ensemble

Pour ce PFR2, l'architecture a évolué pour passer d'une simple simulation à un système réel piloté par une Raspberry Pi communiquant avec une Arduino. Le système repose sur trois piliers :

1. **Le Client (GUI)** : Interface en Python gérant l'affichage et l'envoi des phrases.
2. **Le Serveur (RPI)** : Le cœur en C qui réceptionne, interprète et décide.
3. **Le Microcontrôleur (Arduino)** : Qui exécute les commandes et lit les capteurs.

1.2 Schéma de communication et Flux de données

L'information circule en boucle fermée (aller-retour) à travers deux protocoles de communication :

- **Réseau TCP/IP (Wi-Fi)** : communication asynchrone et sans fil entre le PC et la Raspberry Pi embarquée sur le robot.
- **Liaison Série (UART/USB)** : communication filaire à 115200 bauds entre la Raspberry Pi et l'Arduino.

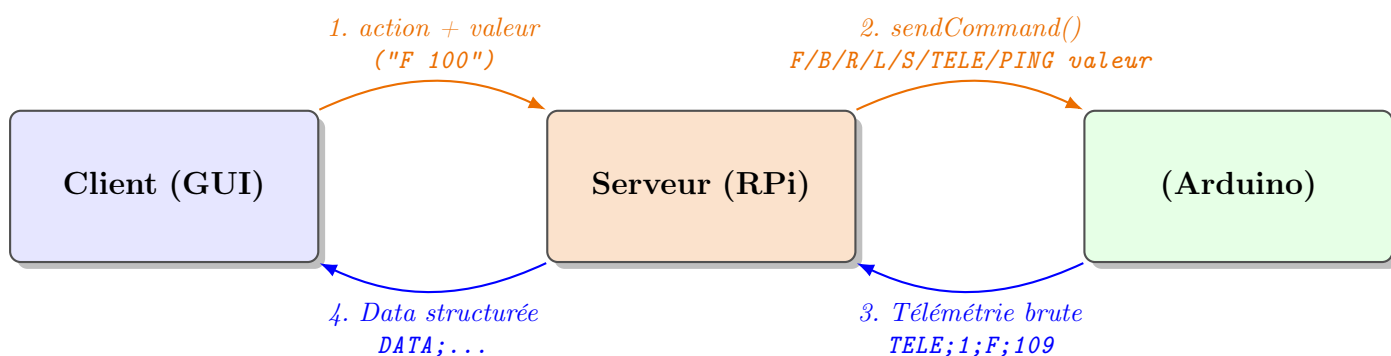


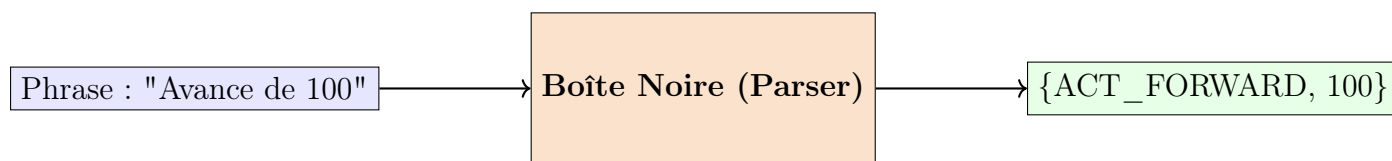
FIGURE 1.1 – Commandes et Télémétrie

1. L'utilisateur déclenche une action sur le GUI. La requête transite instantanément via le réseau TCP.
2. La Raspberry Pi intercepte le message, le valide, et le traduit en instruction machine compréhensible par le microcontrôleur.
3. L'Arduino reçoit l'instruction, l'ajoute à son buffer circulaire et active les moteurs physiques tout en surveillant ses capteurs ultrasons.
4. Toutes les 200 ms, l'Arduino compile son état (mouvement en cours, obstacle détecté, distance) et le renvoie à la Raspberry Pi. Cette dernière re-package l'information pour l'expédier au GUI, afin de permettre la visualisation du résultat de son action en temps réel.

1.3 La Boîte Noire du Parsing

En PFR1, nous avons une pipeline complexe (Cutter, Vocabulary, Analysis, Parser). Pour le PFR2, j'ai regroupé toute cette logique dans une "**Boîte Noire**" simplifiée.

Le concept : Le reste du programme n'a pas besoin de savoir comment le texte est découpé. Il fournit une phrase brute et reçoit une liste d'ordres prêts à l'emploi.



- **Nettoyage** : Suppression des majuscules et de la ponctuation.
- **Identification** : Recherche des mots-clés dans le dictionnaire (TreeMap).
- **Association** : Liaison entre une action (ex : "Avance") et sa valeur (ex : "100").
- **Sécurité** : Si la phrase n'a aucun sens, la boîte noire retourne une liste vide pour éviter que le robot ne réalise une action inappropriée.

1.4 Modes de fonctionnement

Le système possède deux modes de fonctionnement : le mode MANUEL et le mode AUTONOME. La commande "M 1" est envoyée par le GUI pour passer au mode autonome, et "M 0" pour passer en mode MANUEL (le mode par défaut).

Si l'on reçoit la commande "S" (stop) ou si l'on détecte une perte de réseau, on repasse directement en mode MANUEL par sécurité.

Le mode autonome consiste à envoyer des commandes "F 30" en boucle jusqu'à la détection d'un obstacle à moins de 40 cm ($\text{distAv} < 40$, une donnée retournée par l'Arduino toutes les 20 ms). Dans ce cas, on envoie les commandes "B 30" et "R 90" pour reculer et tourner afin d'éviter l'obstacle.

Chapitre 2

Communication Réseau et Serveur TCP

Pour relier le PC de l'utilisateur (Client) au robot (Serveur), nous avons opté pour le protocole **TCP/IP**. Contrairement à l'UDP, le TCP garantit que chaque commande envoyée arrivera à destination, sans corruption et dans le bon ordre. La communication s'effectue via le port 5000.

Si le programme se bloque dans l'attente d'un ordre du PC, l'Arduino ne reçoit plus son signal (*heartbeat*) et le système entier se retrouve bloqué (*watchdog*).

Pour éviter que le robot ne se bloque, j'ai implémenté une lecture réseau **Asynchrone** grâce au flag `MSG_DONTWAIT`.

S'il y a un message, il le traite. S'il n'y a rien, il passe instantanément à la suite du code.

```
1 int octetslus = recv(client_socket, buffer, BUFFER_SIZE, MSG_DONTWAIT);
2
3 if (octetslus > 0) {
4     traitementCommande(buffer);
5 }
```

Listing 2.1 – Réception non-bloquante sur la Raspberry Pi

Dans le fichier `tcp_server.c` :

- **Création (Socket)** : On effectue la demande en mode IPv4 (`AF_INET`) et TCP (`SOCK_STREAM`).
- **Attachement (Bind)** : On lie cette socket matérielle au port 5000 et à l'adresse de la Raspberry Pi (`INADDR_ANY`) avec la sécurité (`SO_REUSEADDR`) pour pouvoir redémarrer le serveur immédiatement après un crash, sans que Linux ne verrouille le port.
- **Écoute (Listen)** : Le serveur ouvre ses portes et se met en écoute passive.
- **Validation (Accept)** : Lorsque le GUI de l'ordinateur tente de se connecter, le serveur valide la liaison.

Une fois le client connecté au moment du *Accept*, j'applique le flag **TCP_NODELAY**.

Nos commandes étant très courtes (ex : "F 100"), ce flag force l'envoi immédiat de ces petits paquets.

Chapitre 3

Arduino (Machine à états)

Une fois la commande transférée à l'Arduino, ce dernier va la parser pour la séparer en action et valeur, puis remplir un buffer circulaire de commandes. Cela va nous permettre d'exécuter les commandes arrivantes dans l'ordre d'arrivée avant que la commande en cours ne soit terminée, sauf pour la commande "S" (stop) qui s'exécute en priorité pour arrêter le robot.

- **IDLE (arrêt)** : Le robot ne fait rien.
- **MOVING (déplacement)** : C'est ici que le robot exécute les commandes.
- **EMERGENCY** : Si l'on croise un obstacle ($\text{distAv} < 40$), on sort de cet état lorsqu'il n'y a plus d'obstacle.

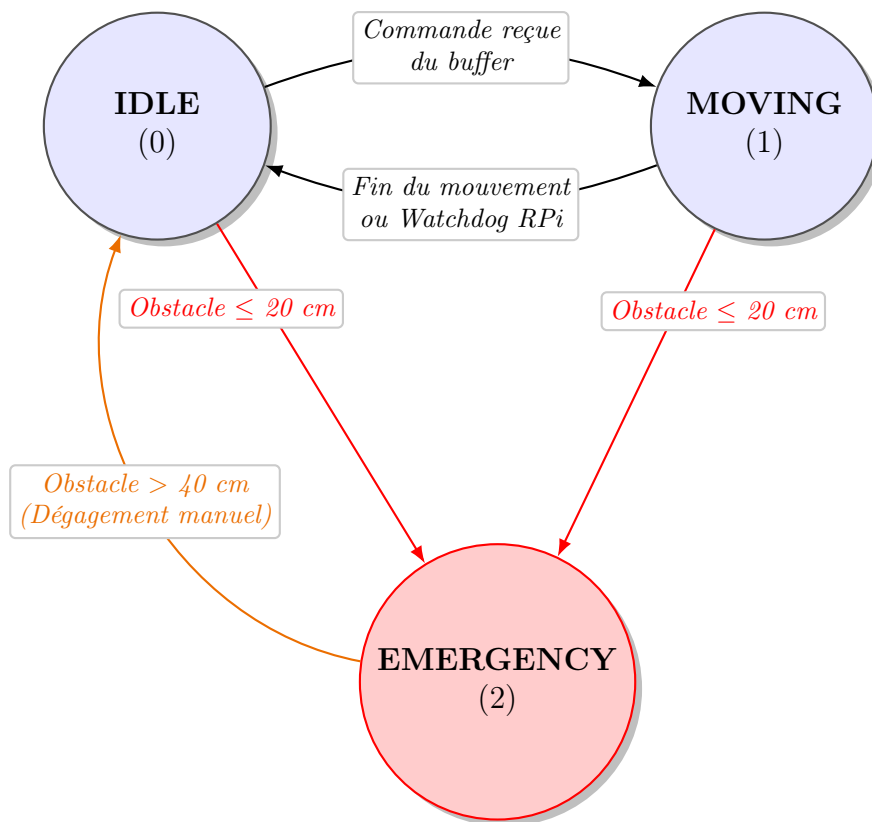


FIGURE 3.1 – Diagramme d'états-transitions du microcontrôleur Arduino

3.1 Sécurité et Watchdog

Le **Watchdog** surveille la connexion entre la Raspberry Pi et l'Arduino. Son but est de s'assurer que la Raspberry Pi est toujours en cours de fonctionnement. Pour cela, j'ai mis en place un **heartbeat** (battement de cœur). Toutes les 50 millisecondes, la Raspberry Pi envoie un message PING à l'Arduino.

L'Arduino possède un compteur de temps. À chaque fois qu'il reçoit un message de la Pi, il réinitialise la variable `dernierMessagePi`. Si le programme de la Raspberry Pi plante ou si le câble se débranche :

- L'Arduino ne reçoit plus de PING.
- Le compteur dépasse une limite de temps (le *timeout*).
- L'Arduino force l'arrêt des moteurs et repasse en état **IDLE** pour éviter tout accident.

3.2 Odométrie Temporelle

Notre robot ne possède pas de codeurs sur ses moteurs. Il est donc impossible de compter les tours de roues pour connaître la distance exacte parcourue. Alors, nous déduisons la distance à partir du temps d'action et d'une vitesse supposée fixe.

$$Temps = \frac{Distance}{Vitesse} \quad (3.1)$$

Par exemple, si l'utilisateur envoie la commande **F 100** (avancer de 100 cm), l'Arduino utilise cette équation pour calculer la durée exacte en millisecondes pendant laquelle les moteurs doivent être alimentés.

Dans notre machine à états, la structure `RobotState` possède une variable `tempsFinAction`.

Au moment où le robot commence un mouvement (état **MOVING**), il lit l'horloge interne de l'Arduino avec la fonction `millis()` et calcule l'heure de fin :

```
1 tempsFinAction = millis() + duree_calculée;
```

Listing 3.1 – Calcul du temps de fin

L'Arduino tourne en boucle et vérifie simplement si le temps actuel a dépassé le temps de fin prévu :

```
1 if (millis() >= tempsFinAction) {
2     nextState.state = IDLE;
3 }
```

Listing 3.2 – Condition d'arrêt

3.3 La Calibration

Le problème est que la vitesse dépend du niveau de la batterie et du poids du robot.

Nous avons donc dû réaliser une **calibration** physique. Nous avons fait rouler le robot sur des distances connues (avec un mètre) et chronométré ses rotations pour définir des constantes de vitesse réelles (en cm/s pour l'avance et en degrés/s pour la rotation). Ces constantes sont corrigées pour obtenir un déplacement correspondant à la distance demandée.

Chapitre 4

Perception et Cartographie : Le Système LIDAR

4.1 Acquisition des données : La bibliothèque RPLidar

Pour l'acquisition des points, nous utilisons un capteur LIDAR piloté par un script Python dédié (`lidar_scan.py`). Ce choix s'appuie sur l'utilisation de la bibliothèque **RPLidar**, qui permet une abstraction de bas niveau du protocole de communication série du capteur.

Cette bibliothèque facilite la récupération des trames de scan sous forme de listes de triplets : (`qualité`, `angle`, `distance`). L'acquisition est réalisée en continu, et les données sont transmises au programme principal en C via un *pipe* de communication.



FIGURE 4.1 – Acquisition de points avec le LIDAR

Le capteur renvoie des données en coordonnées polaires, inexploitable telles quelles pour un calcul de proximité spatiale. Le module `cartography.c` effectue donc un pré-traitement systématique.

4.1.1 Transformation en Coordonnées Cartésiennes

Chaque point polaire $P(r, \theta)$ (où r est la distance et θ l'angle) est converti dans le référentiel orthonormé du robot (x, y) selon les formules trigonométriques suivantes :

$$x = r \cdot \cos(\theta) \quad \text{et} \quad y = r \cdot \sin(\theta) \quad (4.1)$$

Cette conversion est réalisée par la fonction `PolarToCartesian`, permettant de projeter la vision du robot sur un plan 2D euclidien.

4.2 L'Algorithme makeClusters

Une fois les points convertis, l'enjeu est de regrouper les points appartenant au même obstacle physique (mur, pied de table, carton). Nous avons implémenté pour cela un **algorithme de clustering par distance euclidienne**.



FIGURE 4.2 – Regroupement de points en clusters

4.2.1 Logique de regroupement

L'algorithme parcourt le nuage de points. Comme le LIDAR scanne de manière circulaire, les points sont naturellement triés par angle. Pour chaque point P_i , on calcule sa distance par rapport au point précédent P_{i-1} en utilisant la formule de la distance euclidienne :

$$d(P_{i-1}, P_i) = \sqrt{(x_i - x_{i-1})^2 + (y_i - y_{i-1})^2} \quad (4.2)$$

- **Si** $d \leq d_{max}$: Le point est ajouté au cluster courant. On considère qu'il appartient au même objet.
- **Si** $d > d_{max}$: Un "saut" est détecté. L'objet actuel est clôturé et un nouveau cluster est initialisé pour les points suivants.

Le seuil d_{max} (`distance_max`) a été fixé à **200 mm** après plusieurs tests en conditions réelles, permettant de distinguer deux objets proches tout en ignorant le bruit de mesure du capteur.

4.3 Analyse et Reconnaissance de Formes

Une fois les clusters formés, la fonction `recognizeShape` analyse la structure interne de chaque groupe.

1. **Calcul du centre** : La fonction `getCenter` calcule le barycentre du cluster pour positionner l'obstacle sur la carte.
2. **Analyse de courbure** : En comparant les vecteurs formés par trois points consécutifs (A, B, C), on calcule l'angle de courbure à l'aide de la fonction `atan2` sur le produit en croix et le produit scalaire des vecteurs $\vec{AB}$ et $\vec{BC}$.
3. **Classification** :
 - Une courbure globale faible définit une **ligne** (mur).
 - Une courbure élevée et constante définit une **forme ronde** (poteau).
 - Un changement brusque de courbure définit un **angle** (coin).

4.4 Caméra et reconnaissance d'images

Afin d'obtenir les images de la caméra nous avons opté pour la librairie `LibCamera` car il semblerait que cela soit la librairie favorisée pour l'utilisation de caméras sur Raspberry. Le plan initial était de récupérer des images entre chaque actions, puis d'en extraire des objets avec `OpenCV`. Cependant par manque de temps nous avons finir par ne pas du tout utiliser `OpenCV` et avons essayer de nous rabattre sur les codes fait au PFR1.

4.4.1 Fonctionnement de libcamera

Libcamera c'est révéler être une librairie de bas niveau, elle fonctionne sur un système de callback similaire a QT (inspiration direct). Pour l'utiliser il faut d'abord attacher le processus a une caméra matériel, puis on configure la caméra pour nous envoyé le format souhaité, ici nous avons choisis du 640x480 afin de limité la taille des images et nous avons demander a la caméra de nous envoyé du `XRGB888` afin d'évité les conversions inutiles.

Une fois la caméra attaché il suffit juste de créer un tableau de `FrameBuffer` dans le quel libcamera ira stocker les frames a mesure qu'ils les captures. Enfin l'utilisateur vas venir attacher sa fonction de callback a la caméra afin qu'elle soit appliquer a chaque frames capturée. L'exécution du callback est automatiquement faite dans les threads de libcamera (multithread par défaut) mais nous avons tout de même une garantie d'ordre.

4.4.2 Problématique de libcamera

La librairies libcamera étant prévus pour une utilisation réel, elle suppose que l'utilisateur vas faire du traitement vidéo en utilisant un GPU. A cause de cela il est plus facile de capturé en continue que sur demande. Un autre soucis ce présente lorsque l'on souhaite traité les images dans le CPU puisque libcamera fait une capture matériel et n'expose que l'adresse matérielle en supposant qu'un driver GPU viendra la capturé, cela est donc a

faire. Il faut aussi s'assurer que la connexion a la caméra soit correctement fermé a la fin du programme pour évité le risque de blocage matériel.

4.4.3 Solutions et architecture du module caméra

La résolution de ces deux soucis est en vérité plutôt directe. Puisque la librairie est prévus pour une utilisation en C++ nous avons simplement créer une classe qui vas ce charger de la gestion de ressources (**RAII**), de la configuration initiale ainsi que de démarré la capture. A l'intérieur de cette classe ce trouve une méthode Statique qui est donnée a libcamera comme callback et viens extraire le buffer de pixel en mappans l'adresse donnée par libcaméra directement a l'intérieur de la mémoire virtuelle de notre programme (**mmap**).

Puisqu'il s'agit techniquement d'une assignation mémoire il serais préférable d'utiliser un pointeur intelligent pour respecté le **RAII** d'ou la création de la classe **MappedImgBuffer**. Afin d'éviter des bugs imprévus nous avons bien évidemment appliqué la règles des 5 et en particulier nous avons interdie la copie des objets de cette classe pour évité le coût d'une copie complète d'une frame, le constructeur de mouvement est cependant implémenté. La méthode **getPixel** permet de récupéré un pointeur (visiteur) sur la matrice tri-dimensionnelle qui contiens notre image (matrice linéarisée).

Une fois que le buffer a été extrait, il nous suffit plus que de construire notre image de la même manière que lors du PFR1, cependant puisque pour notre robot on aimerais simplement traiter les images sur demande, nous avons décidé de ne pas convertir le buffer directement mais plutôt de le stocker dans un espace mémoire partager entre le producteur (callback de libcamera) et le consommateur (thread qui souhaite obtenir une image), pour cela nous avons simplement utiliser une variable static avec un mutex pour la synchronisation.

Une fois que le la variable static **imgReady** est remplie, n'importe quel utilisateur peut exécuté la méthode **consumeImage** de notre classe **RpiCamera** afin de convertir le buffer en image du pfr1 sur la quelle nous pouvons alors appliquer toute les fonctions déjà construites lors du premier PFR.

Chapitre 5

Multithreading et Architecture Parallèle

5.1 Problématique de la boucle unique (Monothreading)

Initialement, le serveur embarqué sur la Raspberry Pi fonctionnait selon une architecture séquentielle classique basée sur une boucle principale unique. À chaque itération, le programme devait successivement lire les données réseau, acquérir et traiter les points du LIDAR, analyser les images de la caméra, exécuter la logique de contrôle et communiquer avec l'Arduino.

Cette approche a rapidement montré ses limites en situation réelle. Le traitement des données du LIDAR et les algorithmes de vision par ordinateur sont extrêmement gourmands en temps CPU et introduisent des blocages systématiques. Par conséquent, la boucle principale subissait des ralentissements majeurs, entraînant une augmentation critique du temps de réponse du robot. Ce manque de réactivité mettait en péril les fonctions de sécurité, notamment le respect du heartbeat (envoi du PING toutes les 50 ms) et l'arrêt d'urgence face aux obstacles.

5.2 Architecture Multi-threads et passage en C++

Puisque notre robot c'est retrouvé limité par la vitesse d'exécution et que en plus de cela la librairie `libcamera` nous impose le C++, nous avons décidé de monter le niveau d'abstraction de notre programme afin de pouvoir exploiter les types de la librairie standard. Cela a aussi l'avantage de nous permettre d'appliquer le RAII et donc d'éviter un bon nombre de fuites de mémoires.

La charge de travail est répartie de manière asynchrone entre trois threads indépendants s'exécutant en parallèle :

- **Thread Réseau (Communication TCP/IP)** : Il gère exclusivement la socket réseau. Sa mission est d'écouter en continu les commandes textuelles envoyées par l'interface graphique (GUI) et de lui retourner les structures de télémétrie. Isoler cette tâche garantit qu'aucun retard réseau n'impacte le comportement physique du robot.
- **Thread Perception et Capteurs** : Il est dédié à l'acquisition intensive de l'environnement. Il récupère le flux de points bruts du LIDAR à travers le pipe de

communication, exécute l'algorithme de clustering et de reconnaissance de formes, et traite simultanément les images issues de la caméra. Il est également responsable de la construction d'un modèle de l'environnement sur le quel le thread de contrôle pourra raisonner.

- **Thread de Contrôle et Décision** : Il constitue le cœur décisionnel du robot. Il exécute la stratégie de décision souhaitée, traite les ordres reçus du thread réseau, planifie les trajectoires et pilote l'Arduino via la liaison série UART. C'est également ce thread qui maintient le heartbeat de manière régulière.

5.3 Synchronisation et Communication Inter-threads

L'exécution parallèle de ces tâches impose le partage de ressources en mémoire (structures de données de télémétrie, listes d'ordres, coordonnées des obstacles). Pour prévenir les conditions de concurrence (*race conditions*) et les corruptions de mémoire, des mécanismes de synchronisation stricts ont été mis en œuvre.

5.3.1 Structure BiChannel et Files FIFO

La communication entre les différents threads s'appuie sur la classe **BiChannel**. Ce composant implémente deux files d'attente de type FIFO (First-In, First-Out) distinctes :

1. **La file d'entrée (Input Queue)** : Les threads consommateur y pousse des requêtes brutes que le thread producteur vient traiter à son rythme.
2. **La file de sortie (Output Queue)** : Le thread producteur y dépose les données à mesure qu'il les produit afin que les threads consommateurs puissent continuer leurs travaux.

Chaque file FIFO est encapsulée dynamiquement et protégée par son propre mutex, garantissant ainsi l'atomicité des opérations d'insertion (**push**) et d'extraction (**pop**). Cette classe permet ainsi une communication full duplex entre les threads si nécessaire.

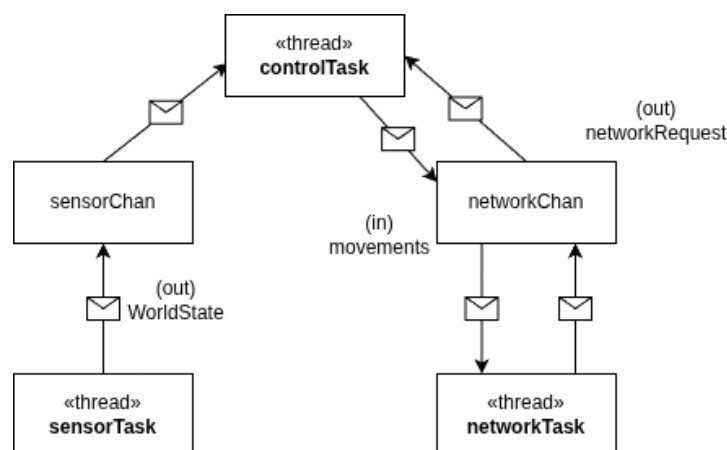


FIGURE 5.1 – Communication entre les différents threads du programme

5.4 Implémentation et Intégration

Puisque le code C++ devait faire l'intégration entre les différents modules venant du C mais que le C++ contient des exceptions il est obligatoire d'utiliser le RAII afin de garantir la bonne libération de la mémoire. C'est pourquoi le code du Raspberry est rempli de classe d'encapsulation qui permette d'utiliser les modules du C sans s'inquiéter de la gestion bas niveau pour se concentrer sur leur unification. Cela permet aussi de s'écarter des détails d'implémentation pour pouvoir se concentrer sur le besoin métier.

5.4.1 Sensor Task

Comme indiqué précédemment ce thread était sensé récupérer les données des différents capteurs et les unifier dans une représentation du monde afin de permettre au système automatique de faire ces décisions ainsi qu'au GUI d'avoir des informations sur le monde autour du robot.

Malheureusement le module de caméra était trop instable pour être intégré correctement et le module Lidar était incomplet. En effet les fonctions C qui permettraient de récupérer une estimation des formes présentes ainsi que de créer la cartographie n'étaient pas encore prêtes et ne pouvaient donc pas être intégrées.

À défaut de pouvoir intégrer les capteurs plus complexes, seuls les capteurs à ultrason ont été utilisés afin de pouvoir au minimum avoir une représentation basique.

5.4.2 Network Task

Cette tâche sert d'interface entre le GUI et le robot, la classe `TcpServer` est celle responsable de cette tâche et de ses fonctions. Le constructeur se charge de démarrer et de configurer le socket serveur comme discuté dans le chapitre 2, la méthode `listen` démarre alors la boucle d'écoute du serveur. C'est ici que les différentes requêtes devaient être traitées et décodées afin de demander à la tâche de contrôle d'atteindre des objectifs. C'est également dans cette méthode que les télémétries devaient être lues depuis le canal d'entrée du `BiChannel` de cette tâche pour être envoyées et affichées dans le GUI.

5.4.3 Control Task

Enfin la tâche de contrôle est le centre de décision du robot. La classe `Pilot` agit comme une interface qui est implémentée soit par `AutoPilot` ou bien `ManualPilot`. Le système a été conçu dès le début pour être changé dynamiquement, cependant l'héritage est inefficace à cause du `dynamic dispatch` et force une structure rigide. Le pattern stratégie est habituellement celui préféré en C++, mais comme il utilise des templates il ne peut pas faire de changement dynamique, c'est pourquoi j'ai opté pour une variation utilisant `std::variant` afin de conserver le `static dispatch` de nos méthodes tout en ayant une description explicite et exhaustive des candidats potentiels et la possibilité d'en changer dynamiquement si l'ordre nous était envoyé.

Le pilote manuel, comme son nom l'indique, n'avait pour objectif que d'exécuter les ordres exacts envoyés par le GUI, son implémentation faisant doublon avec le PFR1 et étant considéré comme moins prioritaire que le mode automatique, il a été mis de côté temporairement.

Le pilot automatique quand a lui implémente la lecture de la représentation de l'environnement puis ce sert de ces informations pour faire une décision. A cause du manque de temps le comportement implémenté c'est retrouvé être extrêmement basique, consistant simplement en un déplacement en avant jusqu'à trouver un mur, puis en une rotation de 90° afin de continuer.

Chapitre 6

GUI - Graphical User Interface

L'interface graphique utilisateur est une section qui fait un lien, entre chaque partie du projet. Celui-ci vas permettre, d'afficher les points du Lidar, différentes données du robot mobile, ainsi que, le fait de pouvoir changer de modes de commande du robot (Manuel, Vocal, Automatique).

Pour le réaliser, plusieurs choix doivent être faits. Le langage de programmation qui a été choisi est le C++, évident par sa simplicité (comparativement au C), son efficacité mais aussi, et surtout, car il nous permet de rester dans l'écosystème C/C++ déjà utilisé. Mais maintenant, il nous faut une librairie adéquate pour la conception d'un GUI. Il nous faut qu'il soit simple à implémenter et rapide. La librairie FLTK pour Fast Light Toolkit est celle que j'ai choisie, car elle respecte tous les critères, facile, rapide, en C++. QT a été aussi étudié mais était certes plus complet mais surtout vraiment plus complexe à implémenter.

6.1 Architecture GUI

Parlons architecture!!

Celui-ci va être composé en plusieurs parties afin de former le tout.

1. **Les Fenêtres**
2. **Les Entités**
3. **Le Canvas de simulation**

Et va aussi impliquer plusieurs autres modules qui seront spécifiés ci-dessous.

1. **Le Client TCP**
2. **La Gestion de données**

6.2 Les différentes Fenêtres

6.2.1 Fenêtre Principal

Toutes les autres fenêtres vont découler de la main Windows. Celle-ci, comme pour toutes les autres, est écrite en classe, et elle va permettre d'afficher toutes les autres, elle est le parent de toutes les autres.

```

1   class SimpleWindow : public Fl_Window { // <-- "le nom de la classe
      ainsi que sa dépendance"
2
3 public:
4   SimpleWindow(int w, int h, const char *title); // <-- "La création
      de la fenêtre"
5   ~SimpleWindow(); // <-- "La méthode de destruction"
6
7   int handle(int event) override;
8   std::unique_ptr<CanvasSim> bot; // <-- "Permet d'appeler le canvas
      de simulation"
9   Client* comm; // <-- "Permet d'appeler le Client"
10 private:
11   };

```

On remarque la ligne permettant d'appeler le canvas de simulation celui-ci utilise un unique pointer car SimpleWindow possède exclusivement l'objet CanvasSim et qu'il doit être détruit automatiquement quand la fenêtre est détruite. Ça permet aussi d'éviter d'oublier de delete donc pas de fuite mémoire, de faire apparaître un double delete. Pour le créer on va utiliser make unique et donc associer l'objet CanvasSim avec ses paramètres.

```

1
2 bot = std::make_unique<CanvasSim>(w-w, h-h, w, h);

```

Il y a aussi la méthode handle qui va permettre la gestion des entrées clavier, plus précisément des touches ZQSD qui vont permettre de faire bouger la représentation du robot sur le GUI ainsi que sur le réel.

```

1
2 int SimpleWindow::handle(int event)
3
4 {
5     if (event == FL_KEYDOWN)
6     {
7         bot->moveFromKey(Fl::event_key());
8         return 1;
9     }
10    return Fl_Window::handle(event);
11
12 }

```

6.2.2 Menu Burger

Le menu burger vas permettre de quitter l'application, de se connecter au serveur, mais aussi d'afficher et de cacher l'overlay. C'est un lien vers les d'autres méthode et fonction

```

1  class BurgerMenu : public Fl_Group {
2  public:
3      std::function<void()> onToggleOverlay;
4      std::function<void()> onQuit;
5      std::function<void()> onConnect;
6
7      BurgerMenu(int x, int y, int w, int h);
8
9  private:
10     Client client;
11     Fl_Button* btn;
12     Fl_Menu_Button* menu;
13
14     static void btnCB(Fl_Widget*, void* self);
15     void openMenu();
16 };

```

j'utilise des lambda expression avec des fonctions callback pour chaque bouton et faire en sorte qu'il y ai des actions quand on clique dessus. Par exemple, pour quitter l'application on vas retrouver la fonction fltk permettant ajouter un bouton et celle-ci attend donc une fonction de callback avec un onclick.

```

1  menu->add("Quitter", 0,
2      [](Fl_Widget*, void* self) {
3          auto* bm = static_cast<BurgerMenu*>(self);
4          if (bm->onQuit) bm->onQuit();
5      }, this);

```



FIGURE 6.1 – burger menu

6.2.3 Overlay

L'overlay permet de voir en temps réel les données du robot que ce soit :

1. **State** : l'état actuel de la state machine
2. **Entities** : le nombre de points du lidar
3. **Current Command** : la commande envoyer en cours

4. **Command Time** : le temps d'exécution de la commande en cours
5. **Front Ultrasound sensor** : la distance en cm de l'ultrason devant
6. **Right Ultrasound sensor** : la distance en cm de l'ultrason à droite
7. **Left Ultrasound sensor** : la distance en cm de l'ultrason à gauche

et pour récupérer les données cela se fait via un canal TCP entre le client (gui) et le serveur (robot).

||



FIGURE 6.2 – Overlay

6.3 Gestion/Affichage des entités

Pour la gestion ainsi que l'affichage des entités je vais utiliser un canvas de simulation qui vas dessiner toutes les entités donc les données reçus du Lidar.

6.3.1 Entités

Pour généraliser les entités il faut créer une classe que l'on va ajouter sur le canvas.

```

1  class GeneEntities : public Fl_Box
2  {
3  public:
4      GeneEntities(const GeneEntities& other)
5          : Fl_Box(other.x(), other.y(), other.w(), other.h())
6      {}
7
8      GeneEntities(GeneEntities&&) = delete;
9      GeneEntities& operator=(const GeneEntities&) = default;
10     GeneEntities& operator=(GeneEntities&&) = default;
11
12     GeneEntities(int x, int y, int w, int h);
13
14     void clearEntities();
15 };

```

Et la vient un problème que j'ai eu je me suis rendu compte que les entités lors de leur création n'arriver pas à ce crée, Je me suis rendu compte alors que je n'avais pas respecte la rules of five du c++ et j'ai donc du ajouter dans la classe j'ai donc ajouter et définit ce que j'utilise. La rules of 5 implique :

1. **Destructeur**
2. **Constructeur de copie**
3. **Opérateur d'affectation par copie**
4. **Constructeur de déplacement**
5. **Opérateur d'affectation par déplacement**

6.3.2 Canvas de Simulation

Le canva de simulation et la fenêtre qui vas afficher toute les entités, le robot, la gestion des touches du clavier pour faire bouger le robot sur la simu, afficher l'overlay.

```

1   class CanvasSim : public Fl_Box
2 {
3 private:
4
5     int inputX;
6     int inputY;
7     int SIZE_RATIO= (w()/h())*30;
8     int BOT_SIZE = 40;
9     int SPEED = 5;
10    std::vector<GeneEntities>obj;
11    void toggleOverlay();
12
13
14
15 public:
16
17    BurgerMenu* burger;
18    DevOverlay* overlay;
19    CanvasSim(int X, int Y, int W, int H);
20
21    void sizeData(std::vector<Polar> Poldata);
22    void draw() override;
23    void moveFromKey(int key);
24 };

```

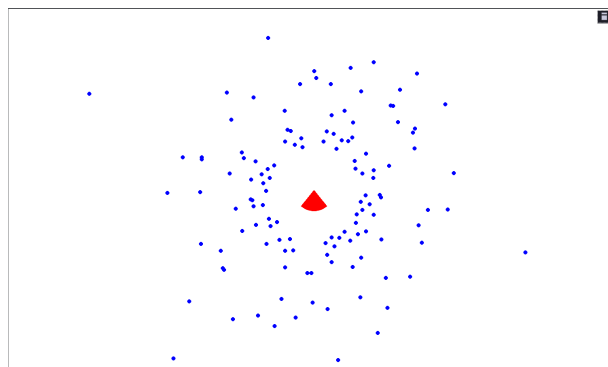


FIGURE 6.3 – main windows

6.4 Client TCP

Pour pouvoir transférer les données entre le GUI et le robot il faut donc se connecter au serveur. La connexion se fait via tunnel TCP et est directement connectée au GUI pour pouvoir lancer la connexion directement.

6.5 Utils

Ici, ce passe tout la gestion des données qui vont être reçues et envoyées au serveur. Ainsi que des outils comme la conversion des données polaire vers cartésien pour pouvoir les afficher proprement sur l'interface.

6.5.1 Gestion de données

Les données que j'envoie et que je reçois je les mets sous forme de struct pour pouvoir les mieux les manipuler .

```
1 struct AppData
2 {
3     State currentState;
4     char currentCommand;
5     int commandTime;
6     int rightDist;
7     int leftDist;
8     int frontDist;
9     size_t entityCount;
10    bool devOverlayVisible = false;
11 }
12 };
13 extern AppData gApp;
```

Chapitre 7

Bilan Personnel

7.1 Bilan de Abdelhafidh GHOUILEM

Ce PFR2 m’a permis de passer de la simulation au matériel. J’ai appris le montage matériel, la connexion de la Raspberry Pi, et l’application de la communication TCP et série (UART). J’ai travaillé sur le transfert de données d’un module à un autre, la gestion des acquittements, des échanges de données et des pertes de connexion, tout en conservant le temps de réponse le plus faible possible. J’ai également appris à rendre un code modulaire et bien documenté, à séparer le cerveau (Raspberry Pi) de l’Arduino, et à mettre en place une machine à états ainsi que des sécurités comme le watchdog avec le heartbeat. De plus, j’ai abordé l’utilisation de buffers, l’introduction au multithreading et l’application des concepts vus en cours cette année (files dynamiques, utilisation de `millis()` au lieu de `delay()`, machines à états, etc.), l’odométrie temporelle avec l’utilisation de l’horloge interne, et la gestion des mouvements du robot.

7.2 Bilan de Victor CHALUMEAUX

Ce projet a été une opportunité majeure pour faire progresser significativement mes compétences en développement informatique, particulièrement en langage C. La nécessité de manipuler des structures de données en temps réel et de gérer rigoureusement l’allocation de la mémoire m’a permis d’acquérir une bien meilleure maîtrise de la programmation bas niveau et de l’optimisation du code embarqué.

Sur le plan technique, la mise en place de la machine à états finis (FSM) directement sur un système matériel réel m’a confronté aux véritables exigences de la robotique autonome. Contrairement à un environnement de simulation, j’ai dû composer avec les bruits physiques et les incertitudes des capteurs pour concevoir un automate capable de prendre des décisions sécuritaires et immédiates face à son environnement.

Enfin, mon travail sur le traitement des données du LIDAR a constitué un défi algorithmique particulièrement formateur. Concevoir la pipeline de transformation géométrique et l’algorithme de clustering m’a appris à transformer un flux de données brutes et complexes en informations spatiales structurées et exploitables par le robot. Ce projet valide ainsi de manière concrète ma capacité à lier perception logicielle, traitement de données et actionneurs physiques sur une plateforme robotique complète.

7.3 Bilan de Joan BELUSCA

Ce fut un énorme projet vraiment très complet et intéressant, je suis partie d'un niveau de connaissance basique en c++ et j'ai sentie une nette amélioration avec ce langage. J'ai du apprendre a crée une interface via une librairie que je ne connaissait pas du tout et surtout toute l'implémentation la recherche de solution tout les problèmes que j'ai pus rencontrer. Malgré un sentiment un peux d'inachevés je suis vraiment content de ce que nous avons produit avec toute l'équipe et nous somme plutôt d'accord sur les points d'amélioration, si ce n'est que personnellement je ressent d'avoir en comparaison avec la première partie réussi a bien évoluer en terme d'efficacité et de connaissance. J'ai vraiment eu l'impression de travailler sur un projet pousser et hyper intéressant et m'as permis de faire preuve d'énormément de créativité et me creuser la tête sur l'agencement de tout les composants les un avec les autres.

Pour conclure, que ce soit les deux PFR, j'ai trouver le fait d'être libre nous permet de complètement faire preuve de créativité et d'inventivité tout du long et surtout c'était une source d'apprentissage et de perfectionnement très efficace car le fait de l'appliquer concrètement sur du physique nous permet de nous rendre compte de nos avancés mais aussi de nos erreurs. J'en ressort grandit et avec beaucoup de points a améliorer pour les futurs projets en équipe ou personnel.

7.4 Bilan d'Arthus DORIATH

Cette seconde partie du projet fil rouge a été pour moi a la fois une source d'apprentissage mais aussi et surtout un grand révélateur de mes lacunes. Mes performances ont été insuffisantes et je ne peut qu'en ressortir avec une grande frustration. Je n'ai pas su être a la hauteur de mes propres standard et mes process habituels ce sont révélés inappropriés a un travail en équipe efficace. Pire que cela, mon mauvais choix de librairie et mon analyse trop lente de la documentation m'ont fait perdre un temps précieux. Tout cela a révéler un besoin critique de réorganiser mes process vers un système d'intégration continue plus moderne ainsi que du besoin de développer une vraie méthodologie d'analyse de documentation afin de trouvé le nécessaire plus rapidement. Je ne laisserais pas cela se reproduire.